

Git と GitHub

1. バージョン管理の必要性

バージョン管理とはあるプロジェクトのファイルやディレクトリーをいつでも元の状態に戻れるように状態を保ちながら編集し続けることをいう。例えば事務的な Excel 書類 hoge.xlsx を

hoge7_4.xlsx

のような形で最終変更日を付けて保存したりしたことがあると思う。しかし、複数の人が同じファイルを扱う場合（自分一人の場合でも）、2つのファイル

hoge7_6.xlsx

hoge7_7.xlsx

が hoge2026_7_4.xlsx からどの順番に生成されたか分からなくなったような経験のある人も多いだろう。

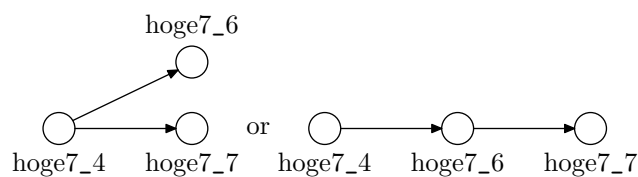


図 1. どちら？

誰が何時手を入れて、それがどのように統合されたのか履歴と責任を明らかにし、そのサポートを行うのがバージョン管理ソフトの仕事である。もちろん、使う側にもその使い方の理解が欠かせない。**Git** はそのようなバージョン管理をするためのツールであり、最近ではバージョン管理のデファクトスタンダードである。

Git ではブランチという枝の様な構造を使ってバージョン管理を行っている。新しいことを付け加えるときはまず幹に当たる main ブランチから新しいブランチを切ってそこで作業し、一段落付いたら main ブランチに統合する、あるいは他の人間にレビューしてもらってその後統合するという方針で作業を行う。

GitHub というクラウドサーバーに適宜ファイルを保存し、ネット上の仲間と共同でバージョン管理するのが今の開発の主流である。

2. バージョン管理ツールの歴史

- SCCS 1972 年 IBM/System 370 向けに開発された。ローカル型。
- RCS 1982 年、ローカル型。
- CVS 1986 年、クライアントサーバー型。
- Subversion 2000 年、クライアントサーバー型。
- Bitkeeper 2000 年、分散型。商用ソフトで Linux の開発でフリーで使われていた。

- Git 2005 年, 分散型. 突然 Bitkeeper が使えなくなったリーナス・トーバルズが開発を始めた.

3. GIT のインストール

Mac や Linux だと既に git がインストールされてるので, ターミナルで

```
% git --version
git version 2.54.0
```

のようにすれば確認出来る. ただし% はプロンプトで入力する必要は無い. 必要に応じて次の行に出力も書いておく. Windows だとインストールが必要.

Git の初期化: まず Git に自分のことを教える必要がある.

```
% git config --global user.name "hoge"
% git config --global user.mail "hoge@example.com"
```

後の GitHub の登録後に本人確認のメールとか届くので, 本物のメールアドレスを登録すること. 細かい設定も似た感じで出来る. 例えば先程言った幹に当たるブランチの名前を main にするには

```
% git config --global init.defaultBranch main
```

とすれば良い (以前は master という名前だったが奴隷制を思い出させるということで main のような名前と呼ばれることが多くなった). なお

```
% git config --list
```

でちゃんと設定できたか確認出来る.

4. GIT の最低限の基本操作

Git ではまずはコマンドラインを使って操作し, 使いながら Git におけるバージョン管理の概念・使用法の理解を深めていくことをお奨めする. 分かってきたらグラフィカルな方法に移って構わない.

- Git はディレクトリ単位でバージョン管理するので, まずディレクトリを初期化する必要がある

```
% mkdir hogehoge
% cd hogehoge
% git init
```

新しいファイルを作って最初のコミットを行う. 最初は説明を省くが後で説明を加える.

```
% echo "Hello Git!" > textA.txt
% git add textA.txt
% git commit -m "first commit"
```

- git の状態を表示

```
% git status
On branch main
nothing to commit, working tree clean
```

- 新しいことをするには、main ブランチでは作業を行わず、例えば branchA というブランチを作り、そこに移って作業する。

```
% git branch branchA
% git switch branchA
```

この段階で書類を VSCode など編集すると、このブランチ branchA の上で編集されるだけで、main ブランチの内容はもとのままに止まっているが、現在の編集内容も保存されていない。
例えば適当にファイルを変更して

```
% echo "add a comment" >> textA.txt
```

- 編集内容を保存するため、ファイル textA.txt をステージング

```
% git add textA.txt
```

し、問題なければコミットして作業中の内容を保存。

```
% git commit -m "edit textA"
```

"edit textA" がそのコミットのコメントで、後で確認出来るように必ず付け加えることになっている。もし git commit のみだとエディターが自動的に立ち上がって、入力を要求される。
以後同じことを繰り返す。

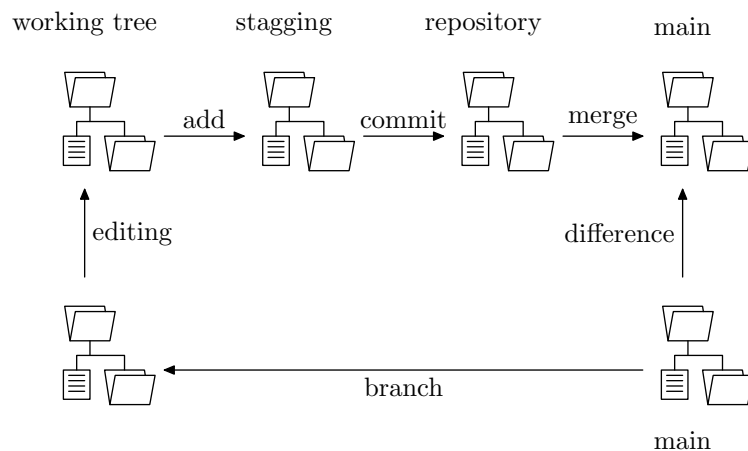


図 2. 作業 branch と main branch

- 幾つものファイルを同時に編集しても構わないし、何度かブランチ branchA 上でコミットを続けることも出来るので適宜部分的に保存しながら作業できる。
- branchA ブランチの変更した内容を main ブランチに付け加えることをマージというが、特に一人で開発しているときはブランチ一個で作業し**ストレートフォワードマージ**といい、何も手を入れずそのまま main にマージすることが多いだろう。
やり方を説明すると、一回メインブランチに移動して

```
% git switch main
% git merge branchA
```

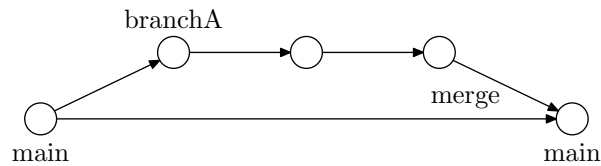


図 3. branchA と Main branch とコミット状況

大事なことなので強調しておく、Git では main で編集作業をすることは推奨されていない。新たな編集作業は新たなブランチで行い、切りの良いとき main に統合することを繰り返すことが奨励されている。

演習. 以上のことをターミナルで行なってください。

5. GIT のやや詳しい操作

以下もう少し詳しいやり方を説明する：

- ブランチの関係を見る

```
% git log
```

情報が多すぎるので各項目を一行で表示

```
% git log --oneline
```

```
011784d (HEAD -> main, origin/main, testSite) add site data
c2b3d4f first commit
```

の方が見やすいかもしれない。最初の数字がコミットハッシュ(の一部)で、この数字でコミットを制御している。関係をグラフで表示することも出来る

```
% git log --graph --oneline
```

- リポジトリの全てのブランチ名の表示

```
% git branch
```

- コミットハッシュ c1f4522 のブランチに移動

```
% git switch --detach c1f4522
```

- ブランチ <branchName> の削除

```
% git switch -d <branchName>
```

適宜不必要なブランチを消して整理することをお奨めする。

- ステージングの取消し

```
% git rm --cached <fileName>
```

- 全ての変更ファイルをステージング：

```
% git add -A
```

- カレントディレクトリの全ての変更ファイルをステージング：

```
% git add .
```

- コミットしたコメントを間違えたときの変更の仕方：

```
% git commit --amend -m "amend text"
```

- 最後のマージとの変更点表示
`% git diff`
- <fileName>の変更点表示
`% git diff -- <fileName>`
- 途中を消すには
`% git merge --squash <branchName>`
- merge で衝突が起こった場合の解消
`% git merge --abort`
 で一回中止し、編集して衝突を除く.
- マージの他に rebase というものもある
`% git rebase`
- main で間違えて作業したときの対応：まず変更点を保存して
`% git stash`
 ブランチを切って
`% git switch -c branch`
 その branch に変更点をアップ
`% git stash pop`

まだ他にも沢山のコマンドがあるので、参考文献などで調べてください。

6. GITHUB

ファイルサーバー GitHub を使うと Git の有用性が広がる。ファイルのクラウドへの保存とネット上でのファイルの共有とかが重要である。個人ではフリーで利用できる。

- **GitHub** を訪れアカウントを作成
- GitHub の自分のページにサインイン
- GitHub の自分のアカウントに ssh で公開鍵を保存：まず自分のパソコンのログインディレクトリーで以下のように公開鍵を作成する。

```
% cd # ホームディレクトリに移動
% ssh-keygen # RSA で秘密鍵と公開鍵の作成
```

途中でパスフレーズを2回聞かれるので（空白でも作れるが）絶対入力するように。上手くいくと

```
% ls -a
...
.ssh
...
```

上のよう鍵が作られている。その後、.ssh のなかの公開鍵を GitHub の自分のホームに登録。

- (1) GitHub の Home の一番右上のアイコンのボタンをクリック (図 4 だと矢印の部分、ファビコンと言うらしい)。
- (2) 左の Setting のボタンをクリック

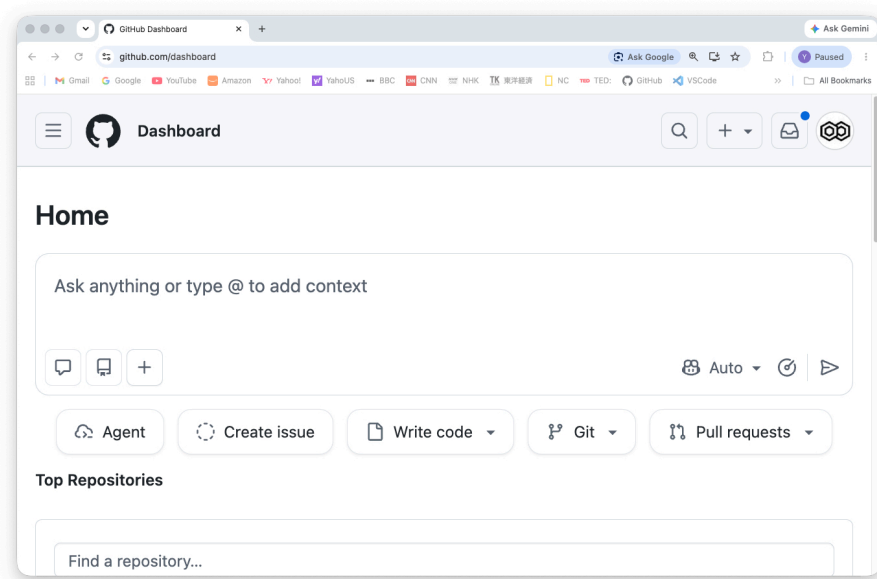


図 4. GitHub の Home

- (3) SSH and GPG Key のボタンをクリック
- (4) SSH の New Key のボタンをクリック
- (5) Title と Key を入力
- GitHubで Repository の New ボタンをクリック Repository Name に (適当な名前)fuga を記入. 他は省略して Create Repository のボタンをクリックするとリポジトリ が作成され, そこに
...or create a new repository on the command line

```
echo "# fuga" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin git@github.com:snow00two/fuga.git
git push -u origin main
```

に従ってコマンドを実行するよう促される. 以下それを解説:
ローカルでディレクトリ fuga を作成し, そこに移動し最低限
必要な README.md を作成

```
% mkdir fuga
% cd fuga
% echo "# fuga" >> README.md
```

そのディレクトリの初期化:

```
% git init
```

README.md をステージングしてコミット

```
% git add README.md
```

```
% git commit -m "first commit"
```

Main ブランチを切り設定

```
% git branch -M main
% git remote add origin git@github.com:snow00two/fuga.git
```

最初のプッシュ:

```
% git push -u origin main
```

ここでパズフレーズを聞かれて、正しければ GitHub にデータが送られる。

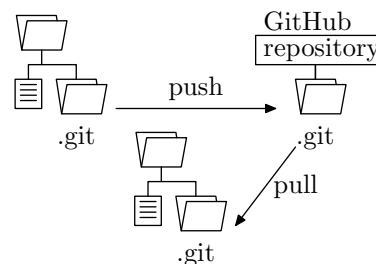


図 5. GitHub, push, pull

- 以後 main ブランチの新しいデータを GitHub のにアップするのは

```
% git push
```

 で良い.
- GitHub からデータを取ってくるには

```
% git pull
```

 これは以下と同じ

```
% git fetch
% git merge
```

 一つずつ確認して作業を行いたい場合はこちらをお奨めする。
 なお最初に GitHub からデータを取ってくるには、例えば

```
% mkdir fractalApp
% cd fractalApp
% git clone https://github.com/snow00two/fractalApp.git
```

 のようにすれば良い.

演習. 以上のことを GitHub とターミナルで行なってください.

7. VSCode と GIT/GITHUB

Git/GitHub と VSCode は相性が良いので、VSCode で Git/GitHub を使うことをお奨めする.

- VSCode で Git で管理されているディレクトリーを開くと自動で Git の管理を感知してターミナルはそのディレクトリーをカレントディレクトリーとして開いてくれる. また. 以前使った

コマンドの History の復元を行ってくれ、コマンドラインから Git や GitHub を管理し易い環境になっている。ファイルの編集自体は VSCode の編集画面を使えば良い。

- VSCode の設定を GitHub で同期するように設定できる。
- GitHub の Student Developer Pack¹ に入っていると、VScode の拡張機能タブで「GitHub Copilot」を検索してインストールすると、使用料無しで VScode から生成 AI の GitHub Copilot が使えるようになり、生成 AI にコーディングの補助を受けながらコーディングすることが出来て便利である。

8. GITHUB PAGES

GitHub では Pages というサービスを使って自分のホームページを世界中に公開できる。

- § 6 で作ったローカルの Repository fuga に docs というディレクトリを作り、その中にホームページとして実現したい html, css, js のファイルを準備する。markdown でも十分で

```
% git branch pages
% git switch pages
% mkdir src docs
% cd src
% echo "# This is my test site." > index.md
% pandoc -f markdown -t html -s -o ../docs/index.html index.md
% git add -A
% git commit -m "add site data"
% git switch main
% git merge pages
% git push
```

- GitHub のホームページの Repository fuga の上の段の Setting(見えないときは More の下に隠れている)を選び、左のコラムの Pages を選択し、セレクトターから Deploy from a branch を選びその下の Branch でセレクトターから main と /docs 選ぶ。

- 暫くすると Setting で Pages の上の部分に

Your site is live at <https://snow00two.github.io/fuga/> のようなアドレスが現れると思う。そこをクリックするとあなたのホームページが表示される (図 6)。markdown については [latex と markdown](#) を参照。

ただもう少し凝ったホームページを作りたい場合これでは不十分なので、[5] や動画などを参考にして HTML や CSS やデザインの基礎を学んで欲しい。

演習. 以上のことを GitHub で行なってください。

¹2026/4 から新規受付を停止していた。いろいろ変わるので詳しいことはネットで確かめてください。

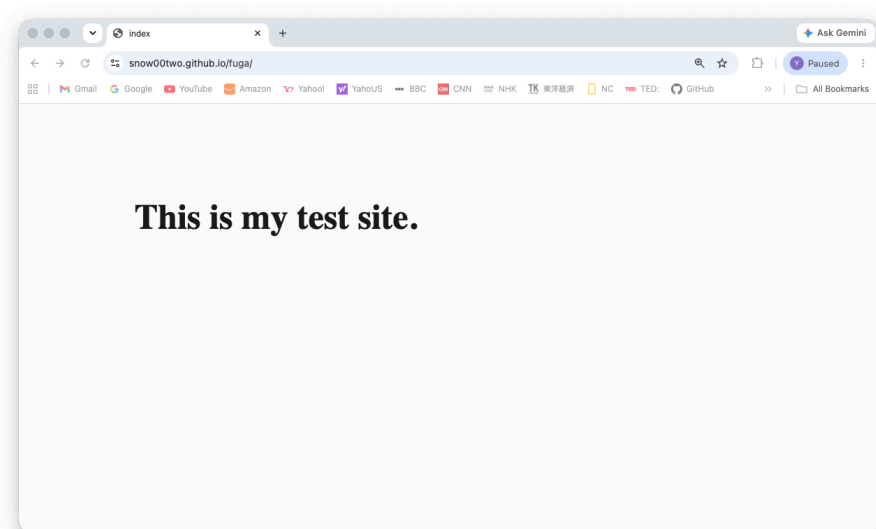


図 6. 作られたホームページ

レポート. 皆さんの作ったアプリや動画や論文を世界に向けて発信するウェブサイトを作って公開し、そのアドレスを私の所に送って下さい。

9. この先に

多人数での開発のためには Pull request と Issue が必要になるが、これについては参考文献などを調べてほしい。また、マージするときに衝突が起こる場合があるのでその場合は手動でマージする必要がある。そのやり方も参考文献などを調べてほしい。

また、Actions という GitHub の自動化のサービスもあるので調べて活用してほしい。

REFERENCES

- [1] サル先生の Git 入門
- [2] 高橋 あおい, つくって、壊して、直して学ぶ Git & GitHub 入門, 翔泳社, 2025.
- [3] 渡辺宙志, ゼロから学ぶ Git/GitHub, 講談社, 2024.
- [4] 実用 Git
- [5] Mana, 1 冊ですべて身につく HTML & CSS と Web デザイン入門講座 [第 2 版], SB クリエイティブ, 2024.